

SETUL DE INSTRUCȚIUNI AL μ P Z-80 (I)

1. Obiectul lucrării

Lucrarea își propune prezentarea setului de instrucțiuni al microprocesorului Z-80. Setul de instrucțiuni este împărțit în mai multe grupe. În această lucrare se prezintă grupa instrucțiunilor de transfer pe 8 biți, cea a transferurilor pe 16 biți și în final lucrul cu stiva. De asemenea, instrucțiunile respective sunt ilustrate cu aplicații.

2. Breviar teoretic

2.1 Limbajul de asamblare

Procesorul înțelege și execută instrucțiuni ce sunt reprezentate în memoria calculatorului în format binar (instrucțiuni scrise cu simbolurile binare “ 0 “ și “ 1 “). Programele scrise în acest format se numesc limbaje mașină (sau cod mașină propriu-zis).

De exemplu, următorul număr binar (sau echivalentul în sistemul hexazecimal), reprezintă codul propriu-zis al unei instrucțiuni :

$$0011\ 1110_{(2)} \quad - \quad 3E_{(H)}$$

Dacă pentru microprocesor acest tip de reprezentare este cel mai adecvat, pentru utilizatorii umani el este impropriu. Este practic imposibil să putem asocia respectivului cod instrucțiunea corespunzătoare (de fapt ce realizează aceasta). Comunicarea între utilizatori cu privire la ceea ce execută microprocesorul este dificilă folosind această reprezentare. Folosirea sistemului de numerație hexazecimal, care reduce de patru ori numărul digiților folosiți în exprimarea informațiilor binare, nu este nici el potrivit pentru exprimarea sensului respectivelor reprezentări.

În vederea mânăuirii facile de către utilizatori a instrucțiunilor procesorului s-a introdus **limbajul de asamblare** denumit și limbaj cu mnemonici. În acest limbaj se asociază unei instrucțiuni o reprezentare prin litere, care de fapt exprimă prescurtat înțelesul acesteia.

De exemplu, pentru instrucțiunea anterioară, prezentată în cod binar (cod mașină propriu-zis), reprezentarea în limbaj de asamblare este:

LD A, n8

Aceste litere provin din prescurtarea LOAD ACCUMULATOR with n8 (încarcă imediat acumulatorul cu constanta pe 8 biți n8).

Se observă că prescurtările provin din limba engleză, fiind propuse de producătorul microprocesorului. Ele trebuie adoptate ca atare fiind folosite astfel

Îndrumar de laborator 2

în programe. Deși la prima vedere pare dificil, totuși utilizatorilor microprocesorului le este ușor să folosească această formă pentru scrierea și înțelegerea programelor.

Un program scris în limbaj de asamblare se numește **program sursă** (nu poate fi “înțeles” direct de microprocesor). Programul corespunzător celui de tip sursă, scris în cod mașină propriu-zis, se numește **program obiect** (va putea fi “înțeles” direct de microprocesor). Trecerea din programul sursă în forma programului obiect se numește **asamblare**.

Asamblarea poate fi realizată manual (folosind tabele de asociații între mnemonică și codul propriu-zis) sau automat (utilizând un program specializat denumit **asambler** sau **crossasambler**).

Limbajul de asamblare permite unui utilizator să aibă în mod direct acces la toate resursele hard ale calculatorului. De asemenea, permite scrierea unor programe foarte rapide și care să utilizeze eficient resursele microsistemului. Dimensiunea acestor programe este minimă din punct de vedere al utilizării memoriei. Din păcate, pentru utilizatori, eficiența în scriere este mică, iar înțelegerea limbajului este dificilă. El nu este accesibil decât unor persoane avizate care posedă cunoștințe legate de arhitectura hardware a microsistemului respectiv.

Un limbaj de nivel înalt (C, Pascal) are pentru un utilizator o eficiență ridicată (doar prin câteva linii în cadrul programului se obțin efecte deosebite). Nu este nevoie de cunoștințe speciale (din punct de vedere hardware) în scrierea acestor programe. De obicei, resursele microsistemului sunt utilizate neoptimal, consumul de memorie fiind mare. De asemenea, accesul la resursele sistemului este îngreunat, iar timpul de execuție al programelor este mărit, neexistând practic nici un control asupra acestuia.

Un program scris în limbaj de nivel înalt este “tradus” procesorului existent în microsistemul respectiv, printr-o operație care se numește **compilare** sau **interpretare**, funcție de tipul limbajului de nivel înalt, iar programul ce efectuează acest lucru este un **compiler** sau **interpretor**.

În aplicațiile considerate critice din punct de vedere al timpului de rulare, precum și de folosire a resurselor microsistemului se recomandă o programare care să îmbine cele două metode (limbajul de nivel înalt și limbajul de asamblare). Partea de program care necesită eficiență crescută din punct de vedere al scrierii și care nu implică probleme critice relative la timpul de rulare și folosirea resurselor microsistemului se scrie în limbaj de nivel înalt. Acolo unde acest lucru se impune trebuie folosit limbajul de asamblare. Orice limbaj de nivel înalt permite ca în interiorul programelor scrise astfel să se introducă porțiuni în limbaj de asamblare. Unul din obiectivele urmărite în desfășurarea lucrărilor de laborator îl constituie însușirea elementelor limbajelor de asamblare pentru diversele tipuri de microprocesoare și elaborarea de programe în aceste limbaje.

2.2 SETUL DE INSTRUCȚIUNI AL $\mu P Z - 80$

Setul de instrucțiuni al microprocesorului Z-80 este împărțit în 16 grupe. Instrucțiunile sunt reprezentate în memoria microsistemelor pe 1 până la 4 octeți. O instrucțiune este formată din codul propriu-zis al acesteia și, opțional, din operanzi. Primul sau primii doi octeți reprezintă codul propriu-zis al instrucțiunii. Următorul sau următorii doi octeți, dacă există, reprezintă operanzii.

I. Grupa instrucțiunilor de transfer pe 8 biți :

Instrucțiunile din această grupă transferă un octet (8 biți) în cadrul execuției, de la o sursă către o destinație. Simbolic, acest lucru se reprezintă astfel:

$$d \leftarrow s$$

unde d - este destinația, iar s - sursa. Destinația și sursa pot fi regiștri, locații de memorie etc., motiv pentru care apar mai multe subgrupe ale acestui tip de instrucțiune. Instrucțiunile din această grupă lasă nemodificat registrul F al indicatorilor de condiții.

LD d,s - (load d with s) - încarcă registrul destinație d cu sursa s .

Simbolic: $d \leftarrow s$ (conținutul sursei s trece în registrul destinație d)

d = registru general pe 8 biți al microprocesorului;

$d \in \{B, C, D, E, H, L, A\}$;

$s \in \{d, n8, (HL), (IX+e), (IY+e)\}$;

$n8$ = constantă pe 8 biți (număr cuprins în gama 00h ..., 0ffh);

e = număr reprezentat în cod complement față de 2 (CC2) pe 8 biți (deplasament).

NOTĂ: O notație de tipul **(XX)** reprezintă **conținutul locației de memorie** adresat de entitatea **XX**, aflată în interiorul parantezei, care semnifică chiar **adresa locației de memorie**. Pentru microprocesorul Z-80, adresa unei locații de memorie este un număr pe 16 biți (cuvânt), iar conținutul unei locații de memorie (data) este un număr pe 8 biți (octet). Întotdeauna entitatea **XX** (care poate fi un registru dublu) trebuie să reprezinte un număr pe 16 biți.

Cele 5 posibilități în cadrul acestei subgrupe sunt:

a) LD r,r (încărcarea unui registru de la alt registru)

Simbolic : $r \leftarrow r$

Îndrumar de laborator 2

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
LD r, r	- - - - -	4	1	0 1 d d d s s s	Dacă A = 1Dh și C = 4Fh LD A,C final : A=4Fh și C=4Fh

d d d = reprezintă codarea destinației
s s s = semnifică codarea sursei

Tabelul de codare (valabil pentru toate instrucțiunile ce conțin aceste coduri) se prezintă mai jos:

d d d (s) (s) (s)	Registrul
0 0 0	B
0 0 1	C
0 1 0	D
0 1 1	E
1 0 0	H
1 0 1	L
1 1 0	(HL)
1 1 1	A

Spre exemplu, instrucțiunea **LD A, C** va avea codul mașină 0 1 1 1.1 0 0 1 (în binar) sau 79h.

OBSERVAȚIE. Din punctul de vedere al notației, în codul mnemonic al unei instrucțiuni de transfer, entitatea aflată în partea dreaptă (sursa) va trece în entitatea aflată în stânga (destinația).

b) LD r, n8 (încărcarea unui registru simplu cu o constantă pe 8 biți, n8).

Simbolic : $r \leftarrow n8$

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
LD r, n8	- - - - -	7	2	0 0 d d d 1 1 0 n8	dacă A = 1Dh LD A, 3Fh final : A = 3Fh

NOTĂ Instrucțiunea **LD B, 7Ah** va avea codul mașină format din octeții 06h și 7Ah (în această ordine).

c) LD r, (HL) - (încărcarea unui registru cu conținutul locației de memorie adresat de HL).

Simbolic : $r \leftarrow (HL)$

Îndrumar de laborator 2

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
LD r, (HL)	-	-	-	-	-	-	7	1	0 1 s s s 1 1 0	dacă HL = 100h și (100h) = F4h LD D, (HL) final : D = F4h

Pentru ca instrucțiunea să se desfășoare corect, în prealabil este necesar să se încarce registrul dublu HL, cu adresa locației de memorie cu care se va lucra. De exemplu, acest lucru se poate face folosind instrucțiunile **LD H, n8** și **LD L, n8** (pentru încărcarea octetului mai semnificativ și respectiv, mai puțin semnificativ ai adresei).

Nu există o instrucțiune de tipul LD (HL), (HL), care ar avea codul 76h. La întâlnirea acestui cod, microprocesorul execută instrucțiunea HALT (oprire).

Exemplu de instrucțiune : LD D, (HL) are codul 0 1 0 1 0 1 1 0 (binar) sau 56h.

d) LD r, (IX+e) - (încărcarea registrului r cu conținutul locației de memorie având adresa IX + e).

Simbolic : $r \leftarrow (IX+e)$

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
LD r, (IX+e)	-	-	-	-	-	-	19	3	DDh 0 1 s s s 1 1 0 e	dacă IX = 200h și (204h) = F4h LD D, (IX+04h) final : D = F4h

(IX+e) = conținutul locației de memorie adresată indexat cu deplasamentul “e”;

e (deplasament) = număr reprezentat în cod complement față de 2 (CC2); este folosit pentru a reprezenta numerele binare cu semn.

În acest mod de adresare, adresa operandului implicat în operație (în cazul nostru, adresa sursei) se găsește prin însumarea algebrică a conținutului registrului index IX cu numărul cu semn “e” (deplasamentul).

Dacă $e < 0$, operandul se află în spatele locației de memorie de referință, a cărei adresă se găsește în registrul index IX (acest lucru înseamnă către adresa 0000h). Dacă $e > 0$, operandul se află în fața locației de memorie de referință, adică către adresa 0FFFFh.

Astfel, prin folosirea deplasamentului (exprimat față de adresa de referință indicată de IX) putem să selectăm 255 de locații (127 către înainte și 128 către înapoi).

NOTĂ. Reprezentarea în CC2 permite transformarea operațiilor de scădere în operații de adunare. De exemplu :

07h - 03h = 07h + 0FDh = 04h ! ! ! (trunchind rezultatul final la doar 8 biți). Pentru această operație, indicatorul de condiții V (overflow) ia valoarea “0”-logic,

Îndrumar de laborator 2

semnificând nedepășirea gamei de reprezentare pe 8 biți. Dacă acest indicator este "1"-logic, rezultatul obținut trebuie corectat ținând seama de depășire și bitul de transport (carry).

e) LD r,(IY+e)

Simbolic : $r \leftarrow (IY+e)$

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
LD r, (IY+e)	-	-	-	-	-	-	19	3	FDh 0 1 s s s 1 1 0 e	dacă IY = 200h și (204h) = F4h LD C, (IY+04h) final : C = F4h

Exemplu : LD A, (IY +23h) COD MAȘINĂ : FDh 7Eh 23h

Practic, toate observațiile făcute la instrucțiunea anterioară rămân valabile și pentru registrul index IY.

OBSERVAȚII. Pentru toate instrucțiunile ce folosesc registrul index IX, la reprezentarea codului mașină apare prefixul 0DDh, iar la cele ce folosesc registrul index IY, prefixul 0FDh.

Nu există nici o instrucțiune care să lege direct regiștri IX cu IY sau IX cu HL sau IY cu HL.

LD d,r - (load d with r) - încarcă destinația d cu registrul r

Simbolic : $d \leftarrow r$ (conținutul registrului r trece în destinația d)

$d \in \{ r, (HL), (IX+e), (IY+e) \}$

Apar următoarele 4 posibilități ale acestei instrucțiuni :

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
LD r, r	-	-	-	-	-	-	4	1	0 1 d d d s s s	dacă IY = 200h (204h) = 0xF4h și C = 0Eh LD (IY+04h), C final : (204h) = 0Eh
LD (HL), r	-	-	-	-	-	-	7	1	0 1 1 1 0 s s s	
LD (IX+e),r	-	-	-	-	-	-	19	3	DDh 0 1 1 1 0 s s s e	
LD (IY+e),r	-	-	-	-	-	-	19	3	FDh 0 1 1 1 0 s s s e	

LD d,n8 - (load d with data n8) încărcarea unei destinații cu constantă pe 8 biți, n8

Simbolic : $d \leftarrow n8$

$d \in \{ (HL), (IX+e), (IY+e) \}$

Îndrumar de laborator 2

Apar următoarele 3 situații posibile :

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
LD (HL) , n8	- - - - - -	10	2	36h n8	dacă HL=100h și (100h)=F4h LD (HL), 0xF4h final : (100h)=F4h HL=100h
LD (IX+e), n8	- - - - - -	19	4	DDh 36h n8 e	
LD(IY+e) , n8	- - - - - -	19	4	FDh 36h n8 e	

LD A,s - (load accumulator from s) - încarcă acumulatorul de la o sursă s

Simbolic : $A \leftarrow s$

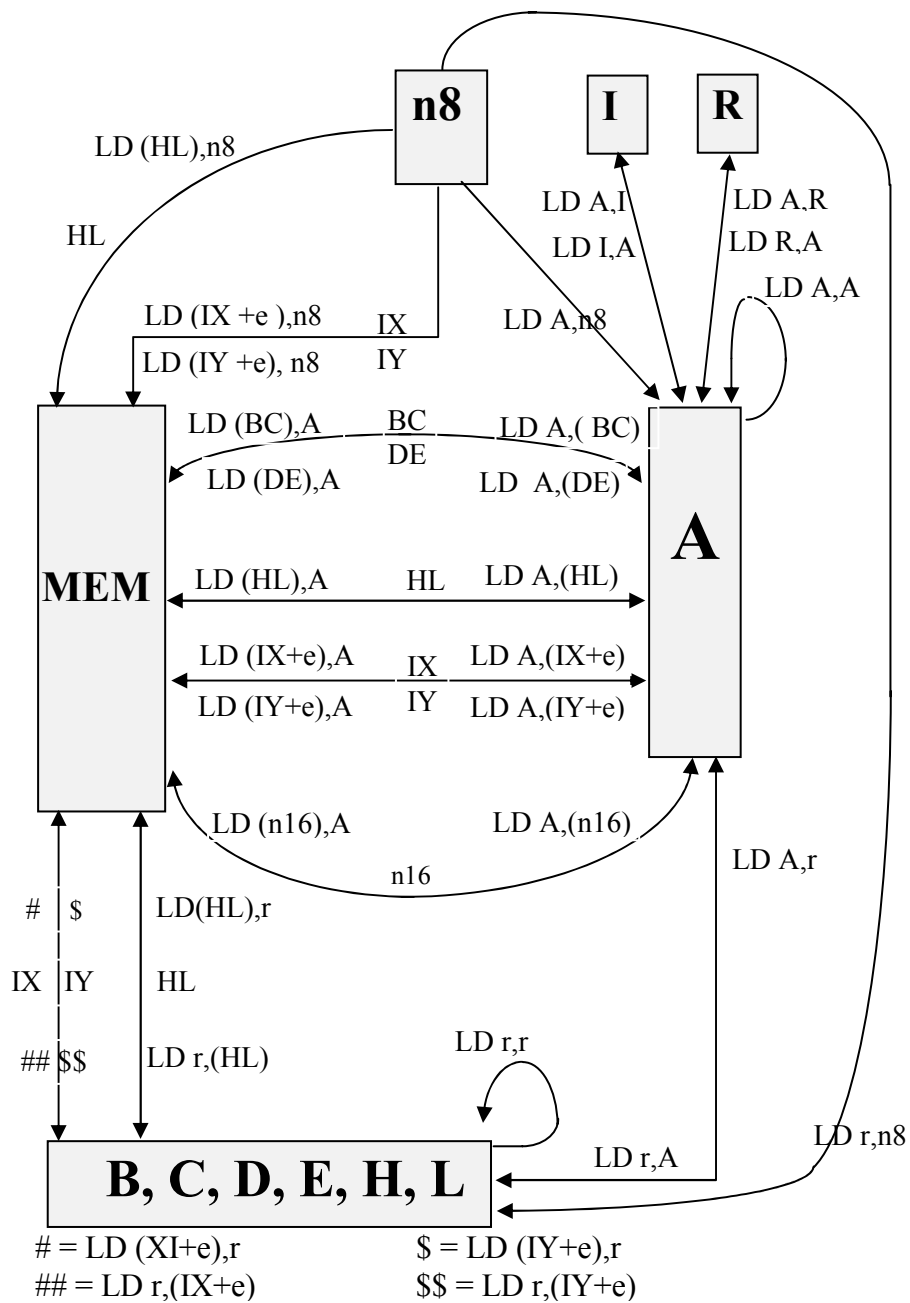
$s \in \{ (BC), (DE), (n16), I, R \}$

Apar următoarele 5 posibilități:

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
LD A , (BC)	- - - - - -	7	1	0Ah	dacă BC=2400h și (2400h)=9Dh LD A, (BC) final : A=0x9Dh (2400h)=9Dh BC=2400h
LD A , (DE)	- - - - - -	7	1	1Ah	
LD A , (n16)	- - - - - -	13	3	3Ah n16 _L n16 _H	
LD A , I	* * 0 * 0 -	9	2	EDh 57h	
LD A , R	* * 0 * 0 -	9	2	EDh 5Fh	

NOTĂ . Reprezentarea unei constante pe 16 biți în memoria microsistemului se face în ordinea ocmps (n16_L) - partea “low”, urmat de ocms (n16_H) - partea “high”. Acest lucru este valabil pentru toate instrucțiunile care operează cu constante pe 16 biți.

OBSERVAȚIE . Ultimele două instrucțiuni reprezintă singurele posibilități pentru citirea și, respectiv, scrierea informațiilor din și în regiștri I și R.



Îndrumar de laborator 2

LD d,A - (load d with accumulator) - transferă acumulatorul în destinația d

Simbolic : $d \Leftarrow A$

$d \in \{ (BC), (DE), (n16), I, R \}$

Apar următoarele 5 posibilități:

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
LD (BC), A	-	-	-	-	-	-	7	1	02h	dacă BC=2400h (2400h)=9Dh și A=0Ch LD (BC), A final : (2400h)=0Ch BC=2400h
LD (DE), A	-	-	-	-	-	-	7	1	12h	
LD (n16), A	-	-	-	-	-	-	13	3	32h n16 _L n16 _H	
LD I, A	*	*	0	*	0	-	9	2	Edh 47h	
LD R, A	*	*	0	*	0	-	9	2	Edh 4Fh	

OBSERVAȚIE . Ultimele două instrucțiuni reprezintă singurele posibilități pentru scrierea și, respectiv, citirea informațiilor în și din regiștri I și R. Toate instrucțiunile de transfer pe 8 biți se pot prezenta unitar în din figura 1.

II. Grupa instrucțiunilor de transfer pe 16 biți :

Instrucțiunile din această grupă transferă în cadrul execuției, un cuvânt (16 biți) de la o sursă dublă către o destinație dublă. Sursa și, respectiv, destinația dublă sunt entități pe 16 biți. Simbolic, acest lucru se reprezintă astfel :

$dd \Leftarrow ss$

dd - semnifică destinație dublă, iar **ss** - sursă dublă. Destinația și sursa pot fi diverse (regiștri pereche, locații de memorie), din acest motiv apărând mai multe subgrupe ale acestui tip de instrucțiune. Instrucțiunile din această grupă lasă nemodificat registrul F al indicatorilor de condiții.

LD dd,n16 - Încărcarea unei destinații duble cu constanta pe 16 biți , n16.

Simbolic : $dd \Leftarrow n16$

dd = destinație dublă

$dd \in \{ BC, DE, HL, SP, IX, IY \}$

Apar următoarele situații :

Îndrumar de laborator 2

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
LD dd , n16	-	-	-	-	-	-	10	3	0 0 rp1 rp2 0 0 0 1 n16 _L n16 _H	LD BC, 2400h final : BC=2400h

LD dd , n16 - realizează încărcarea unui registru dublu cu constanta pe 16 biți, n16 - pentru regiștri dubli BC,DE, HL și SP.

Simbolic: $dd \leftarrow n16$. Instrucțiunea încarcă partea “low” a constantei n16 (8 biți) în registrul mai puțin semnificativ al registrului extins, iar partea “high” a aceleiași constante este încărcată în registrul mai semnificativ al registrului pereche.

Combinăția rp1, rp2 desemnează registrul pereche implicat în operație și este specificată în tabelul următor:

rp1	rp2	registrul
0	0	BC
0	1	DE
1	0	HL
1	1	SP AF

NOTĂ. Prin rp1, rp2 se indică atât sursa cât și destinația. Combinațiile prezentate sunt valabile pentru toate instrucțiunile ce folosesc regiștri dubli.

De exemplu, instrucțiunea LD HL,1234h va avea codul mașină 0 0 1 0 0 0 0 1 (binar) adică 21h și operanzii 34h și 12h (în această ordine). Instrucțiunea provoacă încărcarea în H a constantei 12h , iar în L a constantei 34h. La același efect s-ar fi ajuns folosind două instrucțiuni de transfer pe 8 biți : LD H,12h și LD L,34h .

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
LD IX, n16	-	-	-	-	-	-	14	4	DDh 21h n16 _L n16 _H	dacă IX=2435h LD IX, 60F3h final : IX=60F3h
LD IY, n16	-	-	-	-	-	-	14	4	FDh 21h n16 _L n16 _H	

LD dd,(n16) - Încărcarea unei destinații duble cu conținutul locației de memorie cu adresa n16și respectiv, cu conținutul locației imediat următoare ce are adresa n16 + 1.

Îndrumar de laborator 2

Simbolic : $dd_L \Leftarrow (n16)$ - octetul aflat la adresa specificată prin constanta pe 16 biți, $n16$, trece în partea “ low ” a destinației duble;

$dd_H \Leftarrow (n16 + 1)$ - octetul aflat la adresa imediat următoare, $n16 + 1$, trece în partea “ high ” a destinației duble.

dd = destinație dublă $dd \in \{ BC, DE, HL, SP, IX, IY \}$

NOTĂ. Se atrage atenția asupra faptului că mnemonica instrucțiunii nu exprimă exact ceea ce se execută. Se efectuează de fapt, două transferuri pe 8 biți (sau unul pe 16 biți).

Apar următoarele cazuri:

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
LD dd, (n16)	- - - - -	20	4	EDh 0 1 rp1 rp2 1 0 1 1 $n16_L$ $n16_H$	dacă $IX=2435h$ (04FAh)=77h și (04FBh)=79h LD IX, (04FAh) final : $IX=7977h$
LD IX, (n16)	- - - - -	20	4	DDh 2Ah $n16_L$ $n16_H$	
LD IX, (n16)	- - - - -	20	4	FDh 2Ah $n16_L$ $n16_H$	

LD dd, (n16) - realizează încărcarea unui registru pereche destinație, dd, de la două locații succesive de memorie începând cu adresa pe 16 biți, $n16$ - pentru regiștri BC, DE, HL, SP.

OBSERVAȚIE. Registrul dublu HL nu se supune acestei reguli de codare și, pentru instrucțiunea LD HL, (n16) există următorul cod mașină : 2Ah , $n16_L$, $n16_H$. Explicația excepției constă în faptul că microprocesorul Z-80 a fost creat în idea de a fi compatibil cu un alt microprocesor, creat anterior de firma INTEL, denumit 8080. La acest microprocesor există doar instrucțiunea de echivalență pentru LD HL ,(n16), ceilalți regiștri neavând instrucțiuni echivalente. A fost păstrat codul mașină specificat anterior.

LD IX, (n16) - realizează încărcarea registrului index IX cu conținutul a două locații succesive de memorie.

LD IX, (n16) - realizează încărcarea registrului index IY cu conținutul a două locații succesive de memorie.

LD (n16),ss - Transferarea conținutului unei surse duble, ss, în două locații succesive de memorie, începând cu adresa specificată de constanta pe 16 biți, n16.

Simbolic : $(n16) \leftarrow ss_L$ - octetul aflat în partea “ low ” a sursei duble , se transferă în locația de memorie specificată prin constanta pe 16 biți, n16;

$(n16 + 1) \leftarrow ss_H$ - octetul aflat în partea “ low ” a sursei duble , se transferă în locația de memorie imediat următoare, n16 + 1.

$ss \in \{ BC, DE, HL, SP, IX, IY \}$ ss = sursă dublă

NOTĂ. Se atrage atenția asupra faptului că mnemonica instrucțiunii nu exprimă exact ceea ce se execută . Se efectuează de fapt, două transferuri pe 8 biți (sau unul pe 16 biți).

Apar următoarele cazuri:

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
LD (n16), ss	- - - - -	20	4	EDh 0 1 rp1 rp2 0 0 1 1 n16 _L n16 _H	dacă BC=040Ah (04FAh)=77h și (04FBh)=79h LD (04FAh), BC
LD (n16), IX	- - - - -	20	4	DDh 22h n16 _L n16 _H	
LD (n16), IY	- - - - -	20	4	FDh 22h n16 _L n16 _H	

LD (n16),ss - realizează încărcarea a două locații succesive de memorie, începând cu adresa pe 16 biți, n16, cu conținutul sursei duble ss - pentru regiștri BC, DE, HL, SP.

$ss \in \{ BC, DE, HL, SP \}$

Totuși, registrul dublu HL nu se supune acestei reguli de codare și pentru instrucțiunea **LD (n16), HL** există următorul cod mașină : 22h, n16_L, n16_H . Explicația excepției constă în faptul ca microprocesorul Z-80 a fost creat în ideea de a fi compatibil cu un alt microprocesor, creat anterior de firma INTEL, denumit 8080. La acest microprocesor există doar instrucțiunea echivalentă pentru LD (n16), HL, ceilalți regiștri neavând instrucțiuni echivalente. A fost păstrat codul mașină specificat anterior.

LD (n16),IX - realizează transferarea registrului index IX în conținutul a două locații succesive de memorie.

Îndrumar de laborator 2

LD (n16),IY - realizează transferarea registrului index IX în conținutul a două locații succesive de memorie.

LD SP,ss - Transferarea sursei duble ss în registrul SP, cu păstrarea gradului de semnificație al celor doi octeți ce se transferă.

Simbolic : $SP \leftarrow ss$ $ss \in \{ HL, IX, IY \}$ $ss = \text{sursă dublă}$
Apar situațiile:

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
LD SP,HL	-	-	-	-	-	-	6	1	F9h	dacă SP=0440h
LD SP, IX	-	-	-	-	-	-	10	2	DDh F9h	HL=0C00h
LD SP, IY	-	-	-	-	-	-	10	2	FDh F9h	LD SP, HL final : SP=0C00h

PUSH ss – salvarea în stivă a sursei duble ss.

Stiva este o porțiune a memoriei RAM externă a procesorului. Adresarea stivei se face prin folosirea registrului SP.

Simbolic : $(SP-1) \leftarrow ss_H$
 $(SP-2) \leftarrow ss_L$
 $SP \leftarrow SP-2$

$ss \in \{ BC, DE, HL, AF, IX, IY \}$

Practic, are loc depunerea a doi octeți în două locații succesive de stivă (în ordinea indicată în simbolizarea anterioară), iar registrul SP este reactualizat printr-o dublă decrementare. El indică după această operație, adresa ultimei depuneri în stivă.

OBSERVAȚIE. Orice operație cu stiva trebuie să fie precedată de fixarea bazei stivei prin încărcarea registrului SP cu o constantă pe 16 biți ce exprimă adresa bazei stivei.

Apar următoarele situații:

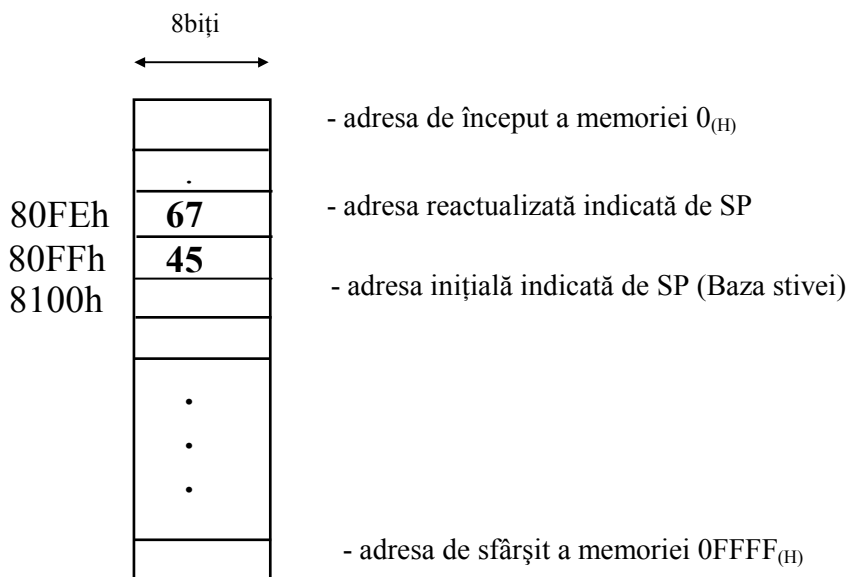
Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
PUSH ss	-	-	-	-	-	-	11	1	1 1 rp1 rp2 0 1 0 1	dacă SP=0440h
PUSH, IX	-	-	-	-	-	-	15	2	DDh E5h	BC=0C0Eh
PUSH, IY	-	-	-	-	-	-	15	2	FDh E5h	PUSH BC final : SP=043Eh (043Fh)=0Ch (043Eh)=0Eh

Îndrumar de laborator 2

NOTA: PUSH ss - (pentru regiștri dubli BC, DE, HL, AF).

APLICAȚIE. Considerând registrul dublu BC și conținutul acestuia 4567h și, respectiv, baza stivei fixată la adresa 8100h, se cere analiza efectului instrucțiunii PUSH BC asupra stivei.

Soluție: Conform structurii instrucțiunii, depunerea se face secvențial (în două etape) - mai întâi se depune în stivă conținutul registrului dublu (partea High - registrul B), apoi conținutul (partea Low - registrul C). Prima depunere se face la adresa de stivă 80FFh, iar a doua depunere la adresa 80FEh. După depunere, registrul SP se reactualizează, indicând adresa ultimei depuneri în stivă, adică 80FEh. Se observă că la o depunere în stivă, SP-ul se decrementează, apropiindu-se de adresa de început a memoriei (stiva crește către înapoi). Pentru zona de memorie implicată, lucrurile sunt prezentate grafic astfel:



POP dd - restaurarea din stivă a destinației duble dd .

Symbolic: $dd_L \Leftarrow (SP)$
 $dd_H \Leftarrow (SP+1)$
 $SP \Leftarrow SP + 2$ $dd \in \{ BC, DE, HL, AF, IX, IY \}$

Practic, are loc extragerea a doi octeți din două locații succesive de stivă (în ordinea indicată în simbolizarea anterioară), iar registrul SP este reactualizat printr-o dublă incrementare. El indică după această operație, adresa ultimei depuneri în stivă.

Apar următoarele situații:

Îndrumar de laborator 2

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
POP dd	- - - - -	10	1	1 1 rp1 rp2 0 0 0 1	dacă SP=043Eh (043Fh)=0Ch (043Eh)=0Eh POP IX final : SP=0440h IX=0C0Eh
POP, IX	- - - - -	14	2	DDh E1h	
POP, IY	- - - - -	14	2	FDh E1h	

NOTA: POP dd - (pentru regiștri dubli BC, DE, HL, AF).

APLICAȚIE. Să se analizeze efectul execuției instrucțiunii POP DE , în condițiile rămase după efectuarea aplicației anterioare (de la instrucțiunea PUSH).

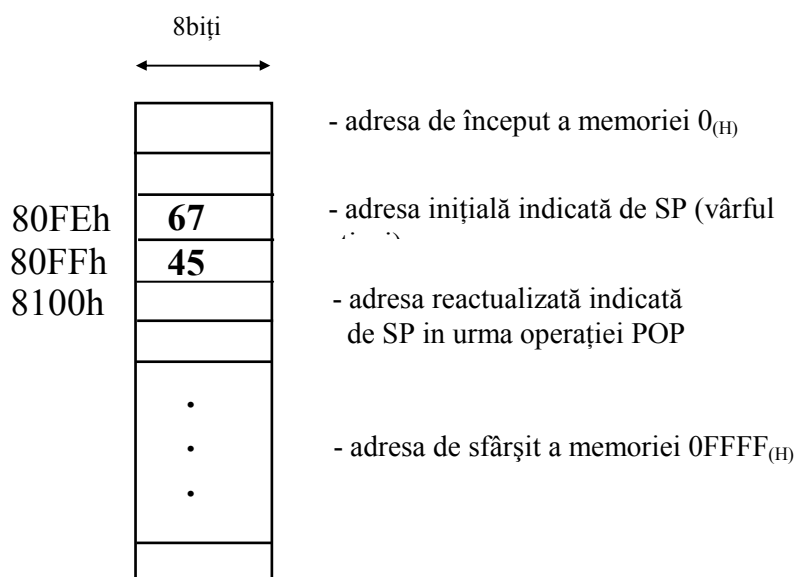
Soluție

Condițiile anterioare rămase sunt:

- adresa inițială a vârfului stivei $SP = 80FE_{(H)}$
- conținutul de la adresa $80FE_{(H)} = 67_{(H)}$
- conținutul de la adresa $80FF_{(H)} = 45_{(H)}$

Derularea operațiilor efectuate de instrucțiune poate fi prezentată astfel:

- Etapa I $E \leftarrow (FE_{(H)})$ adică $E \leftarrow 67_{(H)}$
 Etapa II $D \leftarrow (FE_{(H)} + 1) = (FF_{(H)})$ adică $D \leftarrow 45_{(H)}$
 Etapa III $SP \leftarrow FE_{(H)} + 2 = 100_{(H)}$



OBSERVAȚII

- Instrucțiunile PUSH și POP permit utilizatorului folosirea manuală a stivei, prin intermediul registrului SP ce adresează această parte de memorie. SP-ul se decrementează la depunerile în stivă și se incrementează la extragerile din ea. SP-ul acționează de așa natură încât stiva este adresată prin tehnica LIFO (Last In First Out). Stiva poate fi accesată și automat de microprocesor, în anumite situații speciale (apelarea subrutinelor, tratarea cererilor de întrerupere), situații pentru care în stivă se salvează și, respectiv, se restaurează registrul PC (cazul microprocesorului Z-80).

- Instrucțiunile PUSH AF și POP AF sunt singurele posibilități de citire și, respectiv, scriere ale registrului F, acesta neavând o altă legătură cu ceilalți regiștri ai microprocesorului.

3. Probleme rezolvate

a) Încărcați locația de memorie 9200h cu constanta 55h, locația cu adresa 9201h cu constanta 66h, locația 9202h cu constanta 77h și locația 9203h cu constanta 88h. Se cere ca fiecare transfer să fie realizat printr-o procedură distinctă.

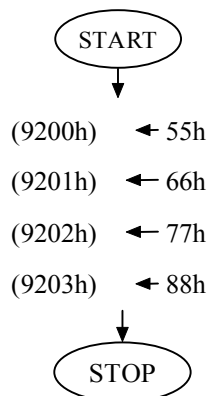
Soluție

Rezolvarea unei probleme în limbaj de asamblare se recomandă a fi făcută în următoarele etape:

1. - elaborarea organigramei problemei; se pleacă de la o organigramă generală urmând un proces de detaliere al fiecărui bloc logic până se ajunge la faza în care este posibilă corespondența cu o instrucțiune (sau grup de instrucțiuni) în limbaj de asamblare; pot fi mai multe etape succesive de detaliere, funcție de complexitatea problemei.
2. - trecerea în limbaj de asamblare prin echivalarea fiecărui bloc logic din organigramă cu o instrucțiune sau grup de instrucțiuni; se obține în acest fel formatul sursă al programului; se va avea în vedere respectarea regulilor de sintaxă ale directivelor programului asambilor (crossasambilor) și, respectiv, ale instrucțiunilor.
3. - trecerea în cod mașină propriu-zis prin asamblarea programului; asamblarea se poate face manual (prin folosirea de tabele de echivalare) sau automat (cu programe de tip asambilor sau crossasambilor); listing-ul rezultat prezintă adresa instrucțiunilor și codul mașină al acestora.

Pentru problema propusă, organigrama generală este următoarea :

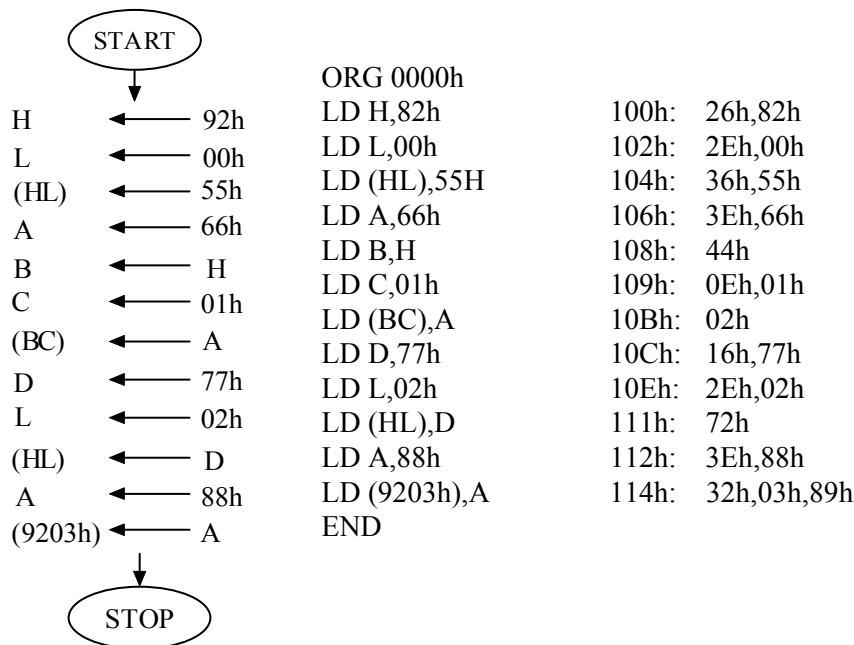
Îndrumar de laborator 2



Această organigramă reprezintă cea mai generală formă de rezolvare a problemei propuse. Într-o a doua fază a primei etape de rezolvare se va urmări detalierea blocurilor logice în vederea ajungerii în faza în care fiecărui bloc îi corespunde o instrucțiune (sau grup de instrucțiuni). Trecerea trebuie făcută având în vedere particularitățile lucrului în limbaj de asamblare la microprocesorul Z-80 și cerințele problemei. Problema propusă este una cu grad scăzut de dificultate și un utilizator experimentat poate trece direct în faza programului sursă fără a mai face detalieri ale organigramei.

Organigrama detaliată, precum și formele sursă și respectiv, obiect ale programului sunt prezentate în continuare. În forma sursă a programului se constată prezența a două directive ale programului asamblor. Este vorba de **ORG 0000h** și **END**. Prima directiva exprimă originea programului obiect (locul în care se va depune în memoria microsistemului, în vederea rulării - în cazul nostru 0000h), iar cea de-a doua indică sfârșitul programului.

Îndrumar de laborator 2



Pentru simularea în mediul de dezvoltare IAR sunt necesari următorii pași:

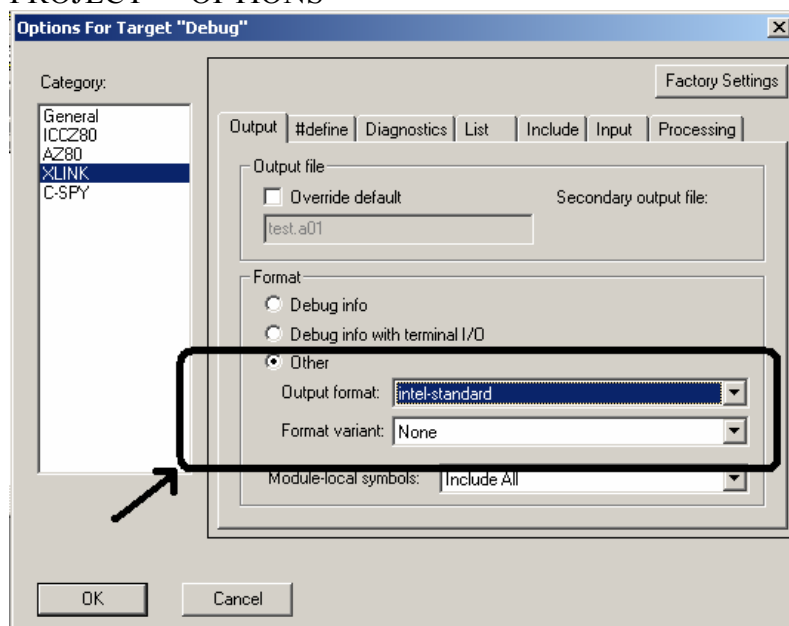
1. Creați un proiect (vezi laboratorul I);
2. Creați un fișier text în care scrieți programul în limbaj de asamblare;
3. Salvați acest fișier cu extensia *asm* și plasați-l în același director cu proiectul după care îl adunați la proiect;
4. După ce ați verificat că este introdus corect, realizați compilarea acestuia prin comanda MAKE;
5. Dacă nu sunt erori, atunci este posibilă simularea acestuia cu programul CSPY care face parte tot din mediul IAR prin comanda DEBUG;
6. Dacă rezultatele programului sunt corecte după simulare, în acest caz îl puteți introduce pe macheta de laborator;

Executarea codului mașină a programului de mai sus pe machetele de laborator este posibilă atunci când codul mașină al acestuia este prezent în memoria de programe a microsistemului. Pentru aceasta, microsistemul a fost prevăzut cu două sloturi de memorie de programe cea ce permite rularea programului într-o zonă de memorie separată de cea a programului MONITOR.

Îndrumar de laborator 2

Pentru implementarea codului mașină pe macheta de laborator este necesară încărcarea acestuia în emulatorul de memorie ROM. Încărcarea programului se realizează cu ajutorul programului SERIAL_SEND.EXE. Fișierul ce conține codul mașină al programului sursă generat de mediul IAR în procesul de asamblare. Pentru obținerea acestuia trebuie realizată modificarea din figura de mai jos.

PROJECT -> OPTIONS



În acest caz, după lansarea operației MAKE, este creat un fișier cu extensia .a01. Acesta este un fișier cu formatul intel-standard ce conține forma binară a programului sursă generat în mediul IAR. Emulatorul de ROM îl va recunoaște, știind să-l transforme iarăși în forma sa binară.

După analizarea programului de mai sus, răspundeți la următoarele întrebări:

- câți ciclii mașină sunt necesari pentru rularea fiecărei instrucțiuni?
- ce valori au regiștri procesorului Z80 după rularea fiecărei instrucțiuni în parte?
- la sfârșitul rulării programului, care este conținutul registrului B ?
- ce modificare se poate realiza pentru a muta valoarea 55h la adresa 9210h si nu la adresa 9200h?
- realizați un program asemănător în care constantele zecimale 10, 45 si 100 sunt depuse la adresele AC00h, AC01h si AC02h.

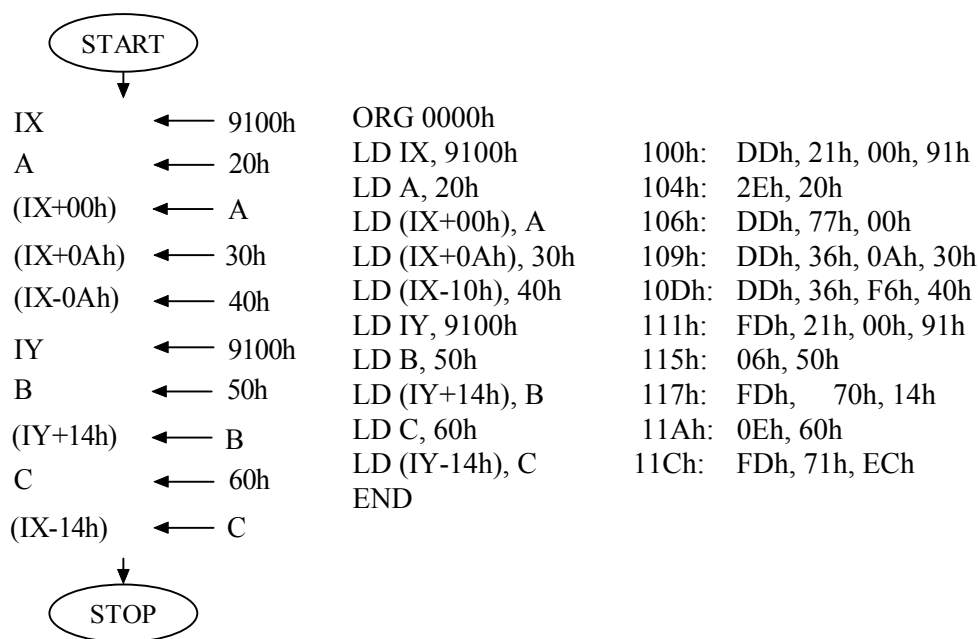
Îndrumar de laborator 2

b) Să se realizeze un program în care se efectuează următoarele transferuri de date utilizând doar regiștri index:

(9100h)	← 20h
(9100h + 10 locatii)	← 30h
(9100h - 10 locatii)	← 40h
(9100h + 20 locatii)	← 50h
(9100h - 20 locatii)	← 60h

Soluție:

După acum s-a arătat în partea teoretică în adresarea indexată se folosește un deplasament care trebuie exprimat în complement față de 2. Așadar valorile 10, -10, 20 și -20 au următoarele echivalențe în CC2: 0Ah, 0xF6h, 14h și respectiv 0xECh. În continuare sunt prezentate organigrama, codul sursă și codul mașină:



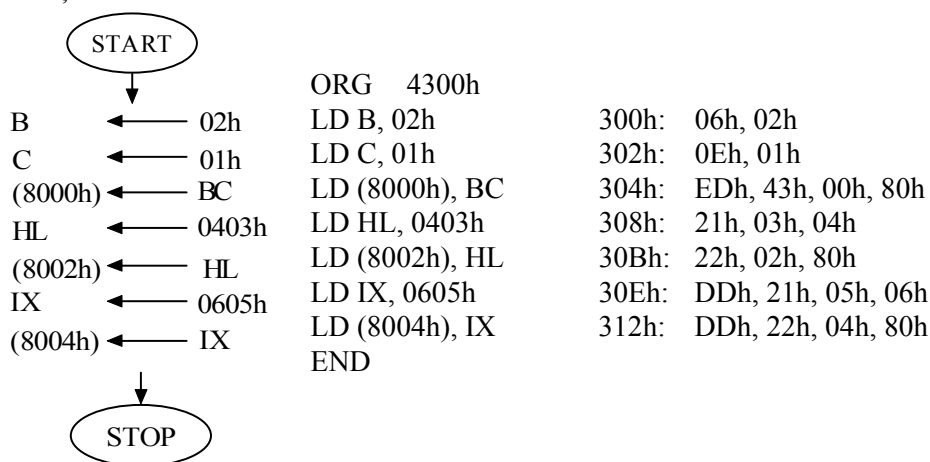
- câți cicli mașină sunt necesari pentru rularea fiecărei instrucțiuni?
- ce valori au regiștri procesorului Z80 după rularea fiecărei instrucțiuni în parte?
- recalculați deplasamentele din enunțul problemei și verificați, după ce rulați programul, dacă octeții respectivi au fost depuși la adresele corecte;

Îndrumar de laborator 2

- realizați un program în care să fie depusă constanta zecimală 23 de la adresa 0A03Dh în 5 locații consecutive dar numai prin modificarea indexului de deplasament.
- modificați programul astfel încât codul mașină al acestuia să fie plasat de la adresa 4000h. Apoi, cu ajutorul programului serial terminal introduceți-l în emulatorul de ROM.

c) Să se realizeze un program în care se completează locațiile de memorie de la adresele 8000h, 80001h, 8002h, 8003h, 8004h, 8005h cu valorile 01h, 02h, 03h, 04h, 05h, 06h. utilizând instrucțiuni de transfer pe 16 biți.

Soluție:



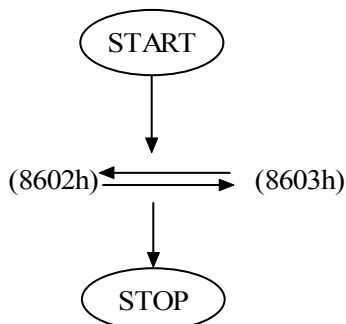
- câți cicli mașină sunt necesari pentru rularea fiecărei instrucțiuni?
- de ce este diferit numărul de cicli mașină la unele instrucțiuni?
- ce valori au regiștri procesorului Z80 după rularea fiecărei instrucțiuni în parte?
- dacă dorim să plasăm constanta hexazecimală 3FC9h începând de la adresa 80D0h, la ce adresa va fi plasat fiecare octet în parte?

d) Realizați un program care să schimbe între ele conținuturile locațiilor de memorie de la adresele 8602h și 8603h.

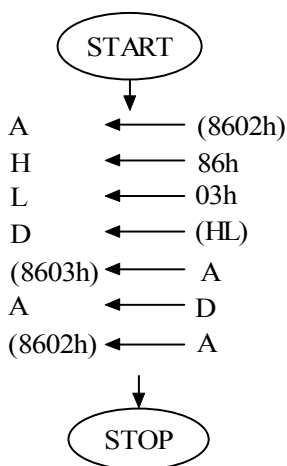
Soluție:

Organigrama generală de rezolvare este:

Îndrumar de laborator 2



O operație directă de schimb între două locații de memorie nu este posibilă în mod direct, printr-o singură instrucțiune. Pentru rezolvare vom folosi un registru liber al microprocesorului (de exemplu registru D), în vederea păstrării temporare a unei informații (de exemplu, cea existentă în locația de memorie cu adresa 8603h. Organigrama detaliată care rezultă, codul sursă și codul mașina sunt :



```

ORG 8200h
LD A, (8602h)
LD HL, 8603h
LD D, (HL)
LD (8603h), A
LD A, D
LD (8602h), A
END
  
```

200h:	3Ah, 02h, 86h
203h:	21h, 03h, 86h
206h:	56h
207h:	32h, 03h, 86h
20Ah:	7Ah
20Bh:	32h, 02h, 86h

- ce modificări apar asupra programului dacă vom scrie ORG 8400h în loc de directiva existentă?
- la sfârșitul rulării programului, care este conținutul registrului L ?
- care este ultima adresă ocupată pentru programul obiect a acestei probleme?
- rezolvați problema printr-o altă modalitate;

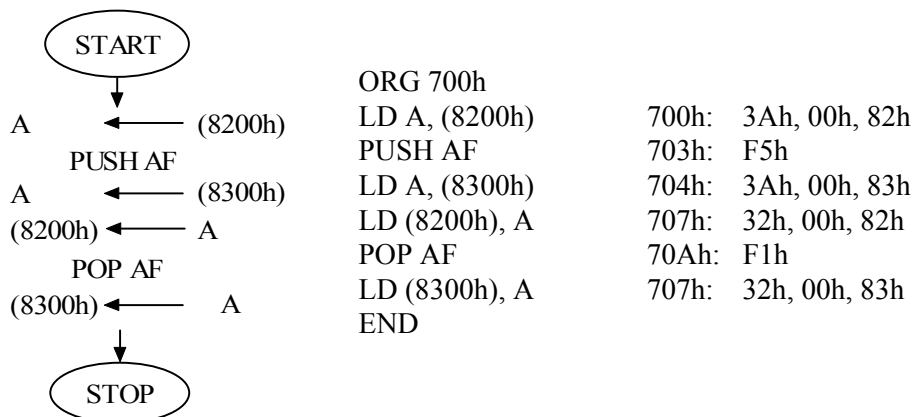
e) Realizați un program care să schimbe între ele locațiile de memorie cu adresele 8200h și 8300h utilizând instrucțiunile de accesare a stivei.

Soluție:

În acest exemplu stiva este folosită pentru salvarea temporară a valorii din locația 8200h. Salvarea în stivă se face cu instrucțiunea PUSH ss, în cazul de față PUSH AF, în A fiind încărcat octetul de la adresa 8200h. Extragerea din stivă s-a

Îndrumar de laborator 2

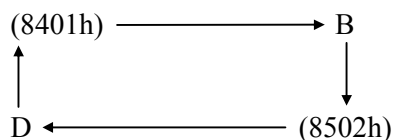
făcut cu instrucțiunea POP AF. În continuare sunt prezentate organigrama, codul sursă și codul mașină al programului:



- ce valori va avea stiva și specificați adresa indicatorului de stivă după rularea fiecărei instrucțiuni.
- plasați baza stivei la adresa 700h și determinați problemele care apar la rularea programului. Care este cauza acestora?

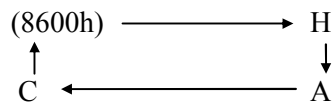
4. Desfășurarea lucrării

1. Se va citi și conspecta breviarul teoretic. Se atrage atenția asupra faptului că toate cunoștințele căpătate în acest laborator vor fi necesare și în derularea celorlalte lucrări.
2. Se vor studia problemele rezolvate, încercând găsirea altor posibilități de rezolvare și se va răspunde la întrebările anexate.
3. Programele obiect ale problemelor rezolvate vor fi introduse în memoria microsistemelor existente în laborator (prin intermediul programului monitor) și se vor rula. Se va observă efectul rulării și buna lor execuție.
4. Se vor rezolva următoarele probleme propuse și se va urmări execuția corectă prin introducerea programelor obiect în memoria microsistemelor și rulare:
 - a). Realizați un program care să efectueze următorul schimb de date prezentat în diagramă:



- b). Realizați un program care să efectueze următorul schimb de date :

Îndrumar de laborator 2



c) Realizați un program care să mute conținutul registrului F în registrul B.

5. Se va răspunde la următoarele întrebări:

a). Ce înțelegeți prin notația “ (1234h) “ ?

b). Există instrucțiunea LD (1234h), (5678h) ?

c). Care sunt exprimările în CC2 pe 8 biți pentru următoarele numere zecimale cu semn :

+47 , -125 , -89 , +120 , -64 ?

d). Explicați efectul rulării instrucțiunilor următoare folosind și reprezentarea grafică:

LD IX , 3456h

LD B, (IX + 0FEh)

e). Se consideră următoarele două forme de instrucțiuni :

$$\left\{ \begin{array}{l} \text{LD H, 56h} \\ \text{LD L, 78h} \end{array} \right. \quad \text{și} \quad \text{LD HL, 5678h}$$

Care sunt asemănările și deosebirile legate de toate aspectele execuției acestor două tipuri de instrucțiuni ?

f). Care va fi conținutul stivei după rularea următoarei secvențe de instrucțiuni:

LD BC, 1234h

LD HL, 5678h

LD SP, 200h

PUSH BC

PUSH HL

g). Cum s-ar putea încărca registrul F cu constanta 00h ?